Measurement Scope Matters: A Validation-First Benchmarking Framework for CPU and GPU AES Implementations

Marko Mihajlovic[1]

Abstract: More than two decades after the standardization of the Advanced Encryption Standard (AES), the growing availability of GPU acceleration raises a practical question: under which conditions can GPUs outperform modern CPUs equipped with dedicated AES instructions? Motivated by the historical transition from DES and Triple DES to AES, this paper presents a validation-first, reproducible benchmarking framework for evaluating AES-128 encryption across heterogeneous CPU and NVIDIA GPU environments. The study compares software CPU implementations, hardware- accelerated AES-NI, and two CUDA implementations in ECB and CTR modes, explicitly distinguishing device-resident kernel execution from synchronous GPU processing that includes host–device transfers and synchronization.

Experiments were conducted on two Windows laptop configurations equipped with 4 GB NVIDIA GPUs (GeForce GTX 1650 Ti and T600 Laptop GPU) and a Linux-based NVIDIA Tesla T4 datacenter configuration. The evaluation comprises 15 independent study sessions and 18,000 timing observations, with paired workloads, cryptographic correctness validation, and independently verified statistical aggregation.

At 32 MiB, GPU resident execution outperformed the corresponding AES-NI CPU implementation in all five sessions on every platform. More importantly, the transfer-inclusive pipeline achieved a 1.35-fold speedup over AES-NI on the T600 Laptop GPU and a 4.76-fold speedup on the Tesla T4 for AES-128 CTR, with the advantage reproduced in all five sessions. However, the consumer GTX 1650 Ti showed no consistently positive pipeline advantage over AES-NI, while the T4 ECB pipeline reached only 0.89 times the corresponding CPU performance. These findings demonstrate that GPU acceleration can outperform hardware-assisted CPU encryption even when data transfers are included, but that the benefit depends on workload size, cipher mode, host CPU, and measurement boundaries.

The study provides reproducible evidence for evaluating GPU offloading on accessible and datacenter hardware and establishes a practical basis for selecting cryptographic execution paths in heterogeneous data-processing and communication systems.

Keywords: Advanced Encryption Standard (AES), AES-NI, GPU acceleration,

[1] Singidunum University, Belgrade, Serbia; ORCID: 0009-0006-7979-0341

CUDA, heterogeneous computing, cryptographic benchmarking, data-transfer overhead, reproducible performance evaluation.

1 Introduction

More than two decades after the standardization of the Advanced Encryption Standard (AES), the efficiency of symmetric encryption remains an important consideration in computing and communication systems. Although the mathematical definition of AES has remained stable, the hardware and software environments in which it operates have changed substantially. Modern processors provide dedicated cryptographic instructions, while graphics processing units (GPUs) offer large numbers of parallel execution resources that can be employed for suitable cryptographic workloads. Consequently, the practical performance of AES depends not only on the algorithm itself, but also on its implementation, the available acceleration mechanisms, data placement, and the operations included in performance measurements.

The historical transition from the Data Encryption Standard (DES), through the Triple Data Encryption Algorithm (TDEA), to AES provides a useful perspective on this development. DES and TDEA played important roles in the evolution of symmetric cryptography, but their limitations eventually led to the adoption of stronger and more flexible alternatives. DES standards have been withdrawn, while TDEA is no longer approved by the National Institute of Standards and Technology (NIST) for applying new cryptographic protection [1–4].

AES, standardized in 2001 and subsequently updated in FIPS 197, remains a fundamental symmetric-key primitive [5]. Its continued relevance reflects not only its cryptographic design, but also its adaptability to changing implementation environments. The original transition from DES and TDEA to AES was motivated primarily by security, efficiency, and implementation requirements. Today, these considerations extend to hardware-assisted execution, heterogeneous computing, and the performance characteristics of complete data-processing systems.

In telecommunications and networked environments, the distinction between cryptographic primitives and their application is especially important. Encryption must coexist with requirements for throughput, latency, authentication, and correct management of cryptographic state. Cipher Block Chaining (CBC), Counter (CTR), and Galois/Counter Mode (GCM), for example, introduce different computational dependencies and security properties [6, 7].

Therefore, high throughput measured for an isolated AES transformation cannot automatically be interpreted as the performance of an authenticated network protocol or a complete communication application. The evaluated cryptographic functionality and measurement boundary must match the intended interpretation

of the result.

1.1 Motivation and Practical Relevance

The increasing availability of GPU acceleration raises a practical question for system designers: can a GPU improve AES processing performance when the host CPU already supports dedicated cryptographic instructions?

This question is particularly relevant for computers equipped with accessible NVIDIA GPUs, including consumer and entry-level professional laptop devices. Such hardware may offer substantial parallel computational resources without requiring a dedicated datacenter accelerator.

However, the availability of a GPU does not automatically imply that transferring an encryption workload to the accelerator will reduce processing time.

Contemporary x86 processors commonly support hardware-accelerated AES execution through the Advanced Encryption Standard New Instructions (AES-NI). These instructions substantially reduce the computational cost of AES operations relative to selected conventional software implementations [8].

A GPU that appears considerably faster than software-only AES may therefore provide a smaller advantage, or no advantage, when compared against an AES-NI-enabled CPU path.

The choice of CPU reference is consequently central to meaningful heterogeneous performance evaluation.

A second consideration concerns the location of the processed data.

A CUDA kernel can execute AES transformations on buffers already resident in GPU memory. However, applications that receive plaintext in host memory and require ciphertext to be returned to the host must also account for the corresponding transfers and execution orchestration.

The cost of these operations can materially change the observed benefit of acceleration.

For applications that process large collections of independent blocks or suitably organized data batches, GPU execution may be attractive. For small, latency-sensitive operations, fixed dispatch, synchronization, and transfer costs may instead favor direct CPU execution.

These observations motivate a benchmarking approach that distinguishes computational throughput from the performance of the complete measured processing path.

The practical objective is not to recommend GPU offloading for every encryption workload, but to establish reproducible evidence for determining when it may be advantageous.

1.2 Background and Research Gap

GPU implementations of AES have been investigated for many years, demonstrating the potential of parallel hardware for accelerating block-cipher operations [9–13].

These studies have examined implementation strategies, parallel processing granularity, memory organization, and techniques for improving GPU execution efficiency.

For example, Iwai et al. investigated AES acceleration using CUDA and demonstrated the relevance of implementation granularity, memory allocation, and overlapping data transfers with computation [11].

Consequently, the influence of data movement on GPU encryption performance is not a previously unexplored problem.

More broadly, Gregg and Hazelwood emphasized that CPU–GPU comparisons can be misleading when they omit the location of application data and the costs of transferring data between processing devices [14].

Lee et al. similarly demonstrated that the apparent performance gap between CPUs and GPUs depends strongly on the optimization quality of the respective implementations [15].

These contributions establish important principles for heterogeneous performance evaluation.

Nevertheless, they do not eliminate the need to assess concrete cryptographic implementations under reproducible and explicitly defined measurement conditions.

A performance result obtained from isolated GPU execution cannot be assumed to represent a host-driven encryption pipeline. Likewise, a speedup measured against conventional software AES cannot automatically be transferred to a comparison against hardware-assisted AES-NI.

Additional uncertainty arises when measurements from different computing environments are interpreted without preserving the relevant hardware, software, workload, and statistical context.

The research gap addressed in this paper is therefore not the absence of previous GPU AES implementations or prior awareness of data-transfer overhead.

Rather, it concerns the need for an integrated, auditable experimental assessment that combines cryptographic validation, paired workloads, a hardware-accelerated CPU reference, explicit GPU timing boundaries, independent study sessions, and source-level traceability within a common analysis.

Such an assessment is particularly useful when examining accessible laptop

GPUs alongside a datacenter GPU, because a positive result on one system may not generalize to another.

The study consequently emphasizes both successful and unsuccessful offloading conditions instead of selecting only the highest observed throughput values.

1.3 Research Objectives and Questions

The objective of this study is to establish and apply a reproducible methodology for evaluating the performance of AES implementations across heterogeneous CPU and GPU execution environments.

The analysis is motivated by the historical evolution of symmetric cryptography from DES and TDEA to AES, but its principal experimental focus is the contemporary relationship between software CPU encryption, hardware-assisted AES-NI, and GPU-based AES execution.

The study addresses three research questions.

RQ1 Under which tested workload sizes, AES modes, and implementation configurations does GPU execution outperform conventional software AES and hardware-assisted CPU AES-NI, and which advantages remain consistent across independent study sessions?

RQ2 How does expanding the GPU measurement boundary from device-resident kernel execution to a synchronous, transfer-inclusive processing pipeline change the magnitude and direction of the observed CPU–GPU performance relationship?

RQ3 How do performance variability, cipher-mode dependence, workload-size effects, and differences among the tested computing configurations constrain the practical interpretation and generalization of GPU acceleration?

These questions distinguish the performance of a cryptographic implementation from the performance of a particular processing workflow.

They also connect the experimental results to practical architectural decisions: whether an AES workload should remain on a hardware-accelerated CPU or be assigned to a GPU under a specified set of execution conditions.

1.4 Methodological Approach and Contributions

The experimental methodology follows a validation-first design in which cryptographic correctness is established before performance measurements are accepted.

Deterministically generated inputs and matching cryptographic parameters support paired CPU–GPU comparisons, while the execution paths are evaluated using separately defined measurement boundaries.

The framework distinguishes conventional software AES, hardware-accelerated CPU AES-NI, device-resident CUDA kernel execution, and a synchronous prepared-buffer GPU pipeline including host–device transfers and synchronization.

These measurements do not represent interchangeable definitions of encryption performance.

In particular, the prepared-buffer pipeline includes a broader execution scope than isolated kernel timing, but does not include every operation required by a complete encryption application.

The principal experimental campaign was conducted on three configurations: a Windows 10 laptop with an NVIDIA GeForce GTX 1650 Ti, a Windows 11 laptop with an NVIDIA T600 Laptop GPU, and a Linux-based Google Colab environment with an NVIDIA Tesla T4.

The first GPU represents a consumer laptop device, the second an entry-level professional mobile GPU, and the third a datacenter-oriented accelerator.

Both laptop GPUs have 4 GB of device memory, making the evaluation relevant to relatively accessible GPU hardware.

The study comprises five sessions per platform, producing fifteen sessions and 18,000 recorded timing observations.

The measurements cover AES-128 encryption in ECB and CTR modes, five workload sizes, conventional software and AES-NI CPU paths, and baseline and optimized-v1 CUDA implementations.

The statistical analysis preserves session-level pairing and reports results with their observed variability. Original timing records, validation outcomes, environment metadata, and implementation source snapshots are retained for independent inspection and reanalysis.

The paper makes three principal contributions:

1. **A validation-first, reproducible benchmarking methodology.**

   The study integrates functional correctness checks, deterministic paired workloads, explicit measurement boundaries, session-based aggregation, and archived experimental evidence into a coherent procedure for evaluating cryptographic execution paths.

2. **A three-configuration empirical assessment of AES acceleration.**

Fifteen study sessions and 18,000 raw timing observations characterize software CPU AES, hardware-assisted AES-NI, and two CUDA implementations under resident and transfer-inclusive GPU execution scopes.

Independent reconstruction of the saved statistical estimators provides additional evidence of the internal consistency and auditability of the reported results.

3. **An evidence-qualified interpretation of GPU offloading.**

   The analysis identifies conditions under which accessible GPU hardware can outperform a corresponding AES-NI CPU reference, while also documenting configurations in which the advantage disappears or reverses.

   Workload size, cipher mode, CPU reference performance, measurement scope, and between-session variability are treated as essential qualifications of practical acceleration claims.

These contributions do not rely on introducing a new AES algorithm, a novel cryptanalytic method, or a claim of record-setting CUDA throughput.

Instead, the contribution lies in applying a consistent and verifiable experimental methodology to practical heterogeneous AES execution choices.

1.5 Principal Findings and Significance

The results demonstrate why the distinction between isolated computation and transfer-inclusive processing is practically important.

For 32 MiB AES-128 workloads, device-resident GPU execution outperforms the corresponding host AES-NI reference in all five sessions on each platform, for both tested CUDA implementations and both encryption modes.

However, this resident advantage does not universally persist when the synchronous GPU pipeline includes data transfers and synchronization.

A particularly relevant positive result is obtained on the NVIDIA T600 Laptop GPU.

For AES-128 CTR at 32 MiB, its baseline CUDA pipeline achieves a median paired speedup of approximately 1.35× relative to its host AES-NI path, with the advantage observed in all five sessions.

The Tesla T4 achieves a larger paired CTR pipeline speedup of approximately 4.76× for the corresponding workload.

Conversely, the consumer GTX 1650 Ti does not demonstrate a consistently positive pipeline advantage over AES-NI, and the T4 ECB pipeline remains slower than its own AES-NI reference at 32 MiB.

These contrasting outcomes show that GPU offloading can provide a measurable benefit even when transfers are included, but that the benefit cannot be inferred solely from GPU kernel throughput or hardware category.

The results also demonstrate that relative GPU speedup must be interpreted alongside the performance of the corresponding host CPU.

For system designers, the implication is that cryptographic acceleration should be evaluated as a workload-placement decision.

Large, parallelizable, or device-resident cryptographic workloads may justify GPU evaluation, whereas host-resident or latency-sensitive processing may favor CPU AES-NI.

These observations are relevant to heterogeneous data-processing and telecommunications systems, although the present measurements do not constitute benchmarks of complete TLS, IPsec, or authenticated-encryption applications.

The broader significance is therefore methodological: the observed advantage of GPU acceleration depends on what is measured, how the workload is organized, and which execution path serves as the reference.

1.6 Paper Organization

The remainder of the paper is organized as follows.

Section 3 describes the experimental platforms, correctness validation, workload definitions, paired measurement protocol, timing boundaries, statistical aggregation, and evidence-preservation procedures.

Section 4 presents the heterogeneous AES-128 results, including absolute throughput, paired speedup, measurement-scope effects, and session variability.

Section 5 interprets the findings in relation to practical GPU offloading, CPU hardware acceleration, data placement, telecommunications applications, and limitations of the tested configurations.

Finally, Section 6 summarizes the empirical findings and the methodological contribution to reproducible cryptographic performance evaluation.

2 Related Work and Research Gap

Research on the acceleration of symmetric cryptography has progressed along two complementary directions. The first investigates algorithmic and implementation techniques for improving AES throughput on parallel hardware. The second addresses the methodological requirements for comparing heterogeneous computing systems, including the choice of CPU reference, data-transfer overhead, timing boundaries, experimental variability, and reproducibility.

These directions are closely related but do not answer the same research question. A highly optimized GPU kernel may demonstrate the computational potential of a particular architecture without establishing that transferring data to and from the device is advantageous for an application. Conversely, a transfer-inclusive comparison may reveal important system-level costs without identifying the maximum achievable performance of the underlying GPU.

The present study builds on both research directions. It evaluates validated AES-128 implementations across three heterogeneous configurations, explicitly separating software CPU execution, hardware-assisted AES-NI, device-resident CUDA processing, and a synchronous prepared-buffer GPU pipeline.

The purpose of this section is to position the study relative to previous AES acceleration research, identify the relevant methodological limitations of performance comparisons, and define the specific research gap addressed by the experimental evaluation.

2.1 GPU Acceleration of AES

The acceleration of the Advanced Encryption Standard (AES) on graphics processing units has been investigated for nearly two decades.

Early studies established that programmable GPUs could exploit the data-level parallelism available in symmetric block-cipher operations [9, 10].

The suitability of a cryptographic workload for parallel execution depends partly on its mode of operation.

Electronic Codebook (ECB) mode permits independent encryption of individual plaintext blocks. Counter (CTR) mode similarly permits independent generation of keystream blocks from distinct counter values.

By contrast, Cipher Block Chaining (CBC) encryption introduces a dependency between consecutive ciphertext blocks, limiting direct parallelization within a single message. CBC decryption permits parallel block-cipher evaluation when the corresponding ciphertext blocks are available [6].

These differences explain why ECB and CTR frequently appear in GPU AES performance studies. ECB remains unsuitable for the protection of general structured application data, however, and its use as a performance benchmark should not be interpreted as a security recommendation.

Manavski [9] demonstrated an early CUDA-oriented AES implementation, contributing to the investigation of programmable GPUs as cryptographic accelerators.

Harrison and Waldron [10] also investigated AES encryption and decryption on GPU architectures, providing early evidence that general-purpose graphics

hardware could support symmetric cryptography.

Iwai et al. [11] subsequently studied the effects of thread granularity and memory placement on AES execution using an NVIDIA GeForce GTX 285.

Their experiments identified a processing granularity of 16 bytes per thread as favorable for the evaluated implementation, achieving approximately 35 Gbit/s.

Importantly, they also investigated overlapping computation and data transfers, reporting approximately 22.5 Gbit/s when transfer time was included.

This work demonstrates that the importance of host–device communication was recognized in early CUDA AES research.

The separation of device computation and transfer-inclusive performance is therefore not introduced here as a previously unknown concept.

Later studies investigated increasingly specialized implementation strategies.

Wang and Chu [12] examined GPU AES encryption and decryption using lookup-table-based transformations and independent AES states assigned to GPU execution threads.

Their work illustrates how representation of the cipher state, memory access, and thread-level organization influence the performance of CUDA implementations.

Tezcan [13] investigated shared-memory optimizations for GPU AES, including techniques designed to eliminate memory-bank conflicts.

The reported AES-128 encryption throughput reached 878.6 Gbit/s on an NVIDIA RTX 2070 Super.

This result illustrates the substantial potential of architecture-aware GPU optimization, but it should not be directly compared with transfer-inclusive throughput measured on different implementations and hardware.

More recent bit-sliced approaches have further investigated the relationship between AES representation and GPU parallelism.

A 2024 study [16] reported high-throughput AES-ECB and AES-CTR execution using bit-sliced implementations, including results on an RTX 3080 and an embedded Jetson Orin Nano GPU.

The study demonstrates the continuing relevance of representation-specific optimizations and parallel counter generation in GPU AES research.

Taken together, these studies establish that GPU AES throughput can be improved substantially through memory organization, parallel granularity, instruction use, and algorithm representation.

Nevertheless, reported performance figures must be interpreted within their respective experimental conditions.

Results obtained using different GPU architectures, AES modes, workload sizes, software implementations, and timing boundaries cannot be treated as directly interchangeable measurements.

The present work does not seek to reproduce state-of-the-art optimized GPU AES implementations or claim record-setting encryption throughput.

Instead, a validated CUDA baseline and a lightweight optimized-v1 candidate are used to investigate how the observed CPU–GPU relationship changes with workload size, cipher mode, computing environment, and measured execution scope.

2.2 CPU Acceleration and Fair Heterogeneous Comparisons

A fundamental issue in assessing GPU acceleration is the choice of CPU reference implementation.

Modern x86 processors commonly provide dedicated Advanced Encryption Standard New Instructions (AES-NI), which accelerate important AES transformations through processor-supported operations [8].

The performance of an AES-NI-enabled implementation can differ substantially from that of a conventional software implementation.

Consequently, an apparent GPU advantage over software AES does not necessarily establish an advantage over a modern hardware-accelerated CPU.

A meaningful CPU–GPU comparison should therefore identify the cryptographic library, hardware acceleration path, execution scope, and relevant threading assumptions.

This concern is not specific to cryptography.

Lee et al. [15] investigated CPU–GPU performance comparisons and demonstrated that large reported GPU speedups can decrease substantially when the CPU implementation is appropriately optimized.

Their findings establish a broader methodological principle: comparisons between heterogeneous architectures are sensitive to the quality of the implementations selected as references.

The present study applies this principle by reporting two distinct CPU execution paths.

The conventional software AES implementation provides a reference for assessing the impact of software-level cryptographic computation.

The AES-NI-enabled implementation provides a stronger hardware-assisted CPU reference for evaluating GPU offloading.

The AES-NI path is used as the principal CPU denominator in the paired GPU speedup analysis.

However, the CPU reference is not presented as the maximum attainable AES performance of the entire host system.

Library choice, mode-specific processing, processor features, instruction dispatch, thread count, and invocation overhead can affect the result.

In particular, a single-threaded library execution path should not be interpreted as equivalent to a fully optimized multithreaded CPU encryption service.

Accordingly, the present results characterize measured implementation paths rather than universal architectural performance limits.

2.3 Measurement Scope and Data-Movement Overhead

Another central methodological issue concerns the operations included within a timed interval.

A GPU application may involve input preparation, host-buffer staging, host-to-device transfer, kernel dispatch, device execution, synchronization, device-to-host transfer, and subsequent application processing.

Whether these operations are included in a benchmark determines the meaning of the reported elapsed time.

Gregg and Hazelwood [14] demonstrated the importance of data location in meaningful CPU–GPU comparisons.

Their work emphasizes that application-level performance cannot be inferred solely from device computation when substantial data movement is required.

CUDA programming guidance similarly identifies minimizing unnecessary transfers and organizing computation and data movement as important optimization considerations [17–19].

CuPy documentation further emphasizes that CUDA operations may execute asynchronously relative to the host, making appropriate device timing and synchronization essential [20, 21].

These considerations are particularly important for cryptographic workloads because the input and output buffers may be maintained in CPU memory rather than resident GPU memory.

A resident kernel measurement characterizes device execution after the necessary input data have been placed on the GPU.

A transfer-inclusive pipeline measurement additionally captures the defined host-driven operations needed to move data through the accelerator.

The two measurements therefore answer different computational questions.

For a sequential GPU pipeline, the elapsed time may be represented conceptually as

$$T_{\text{pipeline}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}} + T_{\text{overhead}}, \tag{1}$$

where the terms denote non-overlapping conceptual contributions from transfer, kernel execution, and other operations within the chosen measurement boundary.

This expression is a conceptual decomposition, not a claim that separately measured component medians must sum to the observed median pipeline time.

The overhead term may include dispatch, synchronization, and relevant host-side operations, depending on the implementation.

Operations outside the timer must also be identified.

In the present study, the measured GPU pipeline starts with prepared pinned host buffers and preallocated device buffers.

The interval includes the defined synchronous H2D–kernel–D2H sequence and the synchronization required for completion.

It does not include initial host staging, buffer allocation or pinning, cryptographic key preparation, or complete application I/O.

Therefore, the term *prepared-buffer pipeline* is preferred over an unqualified claim of complete application-level end-to-end performance.

This explicit boundary makes the comparison reproducible and helps prevent resident-kernel throughput from being misinterpreted as the throughput of a complete encryption application.

2.4 Scientific Benchmarking and Reproducibility

The reliability of performance comparisons depends on experimental design as well as implementation efficiency.

Mytkowicz et al. [22] demonstrated that seemingly minor experimental conditions can introduce systematic bias and substantially alter benchmark conclusions.

Their work highlights the importance of documenting execution conditions and potential sources of measurement interference.

Kalibera and Jones [23] examined rigorous benchmarking under practical time constraints and emphasized the need to account for variability across different levels of experimental replication.

Hoefler and Belli [24] further discussed scientific benchmarking of parallel computing systems, stressing defensible performance reporting and the interpretation of measurement variability.

These studies motivate a hierarchical treatment of benchmark observations rather than indiscriminate pooling of all repeated timing values.

The present study organizes observations into independent study sessions.

Each selected platform contributes five sessions, and each measured group contains twenty recorded timing repetitions following five warm-up executions.

The session constitutes the primary replication unit for statistical aggregation.

CPU–GPU speedups are calculated from paired session-level timing medians and subsequently aggregated across the five sessions.

This preserves the workload pairing and avoids treating nested repetitions as independent experimental sessions.

The framework additionally retains raw observations, derived summaries, validation records, source snapshots, and execution metadata.

An independent offline analysis reconstructed the saved statistical estimators from the archived raw data without detecting numerical discrepancies within the declared comparison tolerances.

This verification supports the internal consistency and auditability of the analysis.

However, successful cryptographic validation does not independently establish the absence of timing bias.

Likewise, matching source hashes do not imply identical compiled binaries or equivalent execution conditions across different platforms.

The methodology therefore distinguishes cryptographic correctness, measurement integrity, repeatability, and the external validity of the resulting performance conclusions.

2.5 Research Gap

The reviewed literature establishes two relevant bodies of knowledge.

The first demonstrates the potential of GPU AES acceleration through parallel execution, memory organization, table-based transformations, bit slicing, and architecture-specific optimizations [11–13, 16].

The second establishes methodological requirements for heterogeneous performance comparisons, including appropriate CPU references, data-movement costs, experimental variability, and reproducible benchmarking practices [14, 15, 22–24].

These principles are already well established.

In particular, previous AES-specific research has considered transfer overhead and overlap between computation and communication [11].

Therefore, the present work does not claim to have discovered data-transfer overhead or introduced the kernel-only versus pipeline distinction.

Instead, the specific research gap addressed here is the need for an integrated, validation-first, auditable empirical comparison that simultaneously includes:

1. A conventional software AES reference and a hardware-assisted CPU AES-NI reference;
2. Validated baseline and optimized-v1 CUDA AES-128 implementations;
3. Matched ECB and CTR workloads across multiple input sizes;
4. Explicitly distinguished resident and transfer-inclusive GPU measurement scopes;
5. Independent study sessions, paired statistical estimators, and reported variability;
6. Preserved raw data, configuration metadata, validation artifacts, and source-level provenance;
7. Observations from two accessible NVIDIA laptop GPUs and a Linux-based datacenter GPU configuration.

The objective is not to demonstrate a universal architectural advantage, but to identify conditions under which particular CPU–GPU performance conclusions are supported by reproducible evidence.

The research questions and contributions are stated in Section 1.3 and Section 1.4, respectively.

The study therefore occupies a distinct position between GPU kernel optimization research and general performance-measurement methodology.

Its scientific value lies in connecting validated cryptographic execution, clearly defined timing boundaries, a strong CPU hardware baseline, and independently auditable experimental results.

This approach is particularly relevant to heterogeneous data-processing and communication systems, where the appropriate cryptographic execution path may depend on data residency, workload size, required functionality, and the acceleration mechanisms available on the host.

The resulting methodology provides a basis for informed architectural assessment rather than a predetermined recommendation to favor either CPU or GPU execution.

3 Methodology

This study adopts a validation-first, reproducible experimental methodology for assessing the performance of symmetric cryptographic implementations across heterogeneous CPU and GPU execution environments. Its central premise is that encryption throughput is not an implementation-independent property of a cryptographic algorithm. Rather, observed performance depends on the selected implementation, cryptographic mode, workload size, execution environment, and the operations included within the measurement boundary.

The methodology combines cryptographic correctness verification, deterministic workload generation, paired CPU–GPU comparisons, explicitly defined timing scopes, session-based replication, and hierarchical statistical aggregation. The resulting framework is evaluated on two Windows laptop configurations equipped with 4 GB NVIDIA GPUs and a Linux-based Google Colab configuration equipped with an NVIDIA Tesla T4.

The experimental contribution is methodological rather than algorithmic. The study does not introduce a new AES primitive or claim to establish the maximum attainable throughput of the tested processors. Instead, it examines how the interpretation of GPU acceleration changes when software CPU execution, hardware-assisted AES-NI, device-resident CUDA execution, and transfer-inclusive GPU processing are evaluated under a common, auditable protocol (Figure 1).

3.1 Experimental Platforms

The heterogeneous study was conducted on three distinct hardware and software configurations. Two systems represent accessible laptop-class NVIDIA GPU environments running Microsoft Windows, while the third provides a Linux-based datacenter GPU configuration.

The first platform combines an Intel Core i5-10300H processor with an NVIDIA GeForce GTX 1650 Ti equipped with 4 GB of device memory. The system runs Windows 10 and represents a consumer laptop GPU configuration.

The second platform combines an Intel Core i7-11850H processor with an NVIDIA T600 Laptop GPU, also equipped with 4 GB of device memory. It runs Windows 11 and represents an entry-level professional mobile GPU configuration.

The third platform uses a Google Colab Linux environment with an NVIDIA Tesla T4 GPU. The recorded GPU belongs to NVIDIA's Turing generation and provides approximately 15 GiB of accessible device memory. The cloud environment exposes an Intel Xeon CPU identified as operating at 2.00 GHz and two logical processors. The exact host CPU model is not established by the archived environment record.

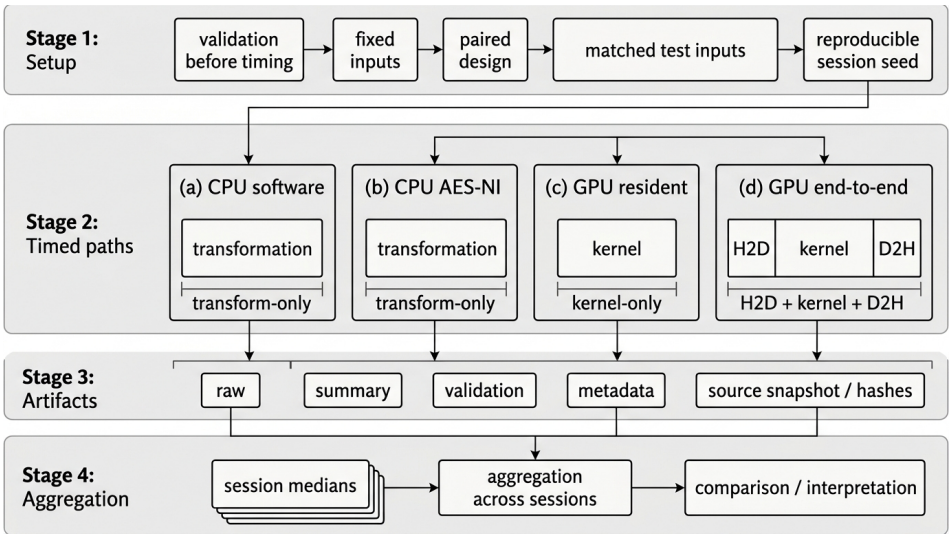


Fig. 1 — Validation-first benchmarking framework. Cryptographic correctness and workload preparation precede timed execution. Paired inputs are evaluated through CPU software, CPU AES-NI, GPU resident, and prepared-buffer GPU pipeline paths. Raw timing observations, session summaries, validation records, metadata, and source snapshots are preserved for subsequent verification and aggregation across independent study sessions. The diagram distinguishes CPU transformation time, device-resident kernel time, and host-observed transfer-inclusive GPU execution.

The principal characteristics are summarized in Table 1.

Table 1
Hardware and software configurations used in the heterogeneous AES study. Operating-system labels describe the tested configurations; kernel and runtime details are derived from the archived environment records.

Platform	**Host CPU**	**GPU and environment**
Windows 10	Intel Core i5-10300H	GeForce GTX 1650 Ti; 4 GB VRAM; consumer laptop GPU; CUDA runtime 12.9; CuPy 13.6.0; PyCryptodome 3.23.0
Windows 11	Intel Core i7-11850H	T600 Laptop GPU; 4 GB VRAM; professional mobile GPU; Turing architecture; SM 7.5
Linux / Colab	Intel Xeon CPU at 2.00 GHz; two exposed logical processors	Tesla T4; approximately 15 GiB VRAM; Turing architecture; SM 7.5; CUDA runtime 13.0; CuPy 14.0.1; PyCryptodome 4.0.0

The CPU implementations and cryptographic reference operations are provided through PyCryptodome [25]. Consequently, the reported CPU measurements characterize specific library execution paths rather than theoretical performance limits of the AES algorithm or the maximum possible performance of independently optimized native implementations.

The GPU implementations use CUDA kernels invoked through CuPy. The CUDA runtime provides kernel execution, memory-management, synchronization, and transfer functionality [17, 21].

The two Windows systems operate within the Windows GPU driver environment, where command scheduling, preemption, and synchronization can influence observed execution behavior [26, 27]. The T4 configuration uses a different Linux and cloud execution environment.

These differences are treated as characteristics of the measured platforms rather than as independently controlled experimental variables. In particular, the study does not isolate the effects of Windows 10 versus Windows 11, operating-system families, GPU product classes, driver versions, or cryptographic library versions.

All cross-platform findings are therefore interpreted as comparisons among three complete hardware–software configurations. Absolute GPU throughput is compared separately from speedup relative to each system's own CPU reference.

3.2 Validation-First Experimental Protocol

Functional correctness is a prerequisite for accepting a performance experiment. Cryptographic implementations are subjected to validation before their timing results are admitted to the principal study.

The validation procedures include known-answer tests, CPU reference comparisons, and consistency checks between supported execution backends. The expected cryptographic behavior is based on the AES specification and the relevant definitions of block-cipher modes of operation [5, 6].

PyCryptodome provides reference operations used in selected CPU–GPU differential checks [25]. The archived validation records additionally document the status of CUDA compilation and the relevant cross-backend correctness checks.

A configuration that fails the required validation procedures cannot be accepted as a successful study measurement.

This requirement is important because a high-throughput implementation is not meaningful as an AES performance result unless the implementation performs the expected cryptographic transformation.

The benchmarking workflow distinguishes three execution profiles:

1. **Smoke profile.** A lightweight execution used to verify basic functionality, cryptographic validation, configuration handling, and the production of required experimental artifacts.

2. **Pilot profile.** A preliminary execution used to inspect measurement behavior, confirm benchmark parameters, and identify operational problems before the principal study.

3. **Study profile.** The controlled, session-based configuration whose validated observations form the primary quantitative comparisons.

Only the designated study-profile observations are used in the principal three-platform performance analysis. Earlier smoke, pilot, developmental, and historical runs are not silently combined with the selected complete studies.

The protocol archives the configuration, random seeds, workload parameters, validation results, timing observations, relevant environment metadata, and implementation source snapshots.

This distinction between correctness and measurement validity is essential. Successful cryptographic validation establishes agreement with expected outputs under the tested conditions. It does not, by itself, prove the absence of timer bias, operating system interference, or other sources of measurement error.

Accordingly, cryptographic validation is complemented by independent verification of experimental completeness, configuration consistency, source identity, and saved statistical estimators.

3.3 Workload Definition and Paired Experimental Design

The principal heterogeneous comparison evaluates AES-128 encryption in Electronic Codebook (ECB) and Counter (CTR) modes.

ECB and CTR are appropriate for controlled examination of independent block-level computation, although their practical security properties and operational requirements differ [6].

ECB is employed solely as a cryptographic performance probe and must not be interpreted as a recommended encryption mode for general application data. CTR measurements characterize encryption throughput under the selected counter configuration; they do not constitute an evaluation of authenticated encryption or application-level counter management.

The five workload sizes are:

$$N \in \left\{2^{10},\ 2^{14},\ 2^{20},\ 2^{24},\ 2^{25}\right\} \text{ bytes.} \tag{2}$$

These correspond to 1 KiB, 16 KiB, 1 MiB, 16 MiB, and 32 MiB.

The selected sizes span small-message execution, intermediate workloads, and larger contiguous buffers for which GPU execution and host–device movement can have materially different relative costs.

Each workload is evaluated through two CPU reference paths:

1. A selected software AES implementation.
2. A selected hardware-accelerated AES-NI implementation.

The CUDA evaluation includes two timed kernel candidates:

1. A baseline CUDA AES implementation.
2. An optimized-v1 CUDA implementation representing a lightweight implementation candidate.

The optimized-v1 implementation is treated as a candidate for empirical comparison, not as an implementation assumed to be superior. Its performance must be evaluated separately for each mode, workload size, platform, and timing scope.

Corresponding CPU and GPU workloads use deterministically generated inputs and matching cryptographic parameters, including the plaintext, key, and mode-specific initialization values.

A fixed seed identifies each study session. This design supports the reconstruction of workload identities and allows CPU and GPU timing comparisons to be paired within the same session.

The five selected seeds are:

$$s \in \{2003, 2004, 2005, 2006, 2007\}. \tag{3}$$

The study uses five warm-up executions and twenty recorded timing repetitions for each measured configuration.

The warm-up phase is excluded from the saved timing summaries. Its execution is supported by the archived configuration and execution provenance, rather than by individual warm-up rows in the raw timing exports.

The workload matrix contains 60 measured groups per session, with twenty recorded observations per group. Each complete session therefore contributes 1,200 raw timing records.

With five sessions on each of three platforms, the primary heterogeneous analysis comprises:

$$3 \times 5 \times 1200 = 18\,000 \tag{4}$$

raw timing observations.

These observations are nested within 15 study sessions, rather than representing 18,000 statistically independent experimental replications.

The execution order follows the archived benchmark protocol rather than a randomized sequence of workload sizes. Consequently, workload size and elapsed experimental time are not completely separated as potential explanatory factors.

The paired design reduces differences arising from unmatched cryptographic inputs, but does not make CPU and GPU host-side operations identical. Their timing boundaries are therefore reported explicitly.

3.4 Measurement Boundaries

A central methodological decision is to define the operations included in each timed interval.

The study distinguishes four principal paths:

1. **CPU software transformation.** Measures the selected software AES en-

cryption operation on the host CPU.

2. **CPU AES-NI transformation.** Measures the selected hardware-accelerated CPU AES operation through the cryptographic library.
3. **GPU resident execution.** Measures CUDA kernel execution with the input already resident in device memory. Host-to-device and device-to-host transfers are excluded from this interval.
4. **Prepared-buffer GPU pipeline.** Measures the defined host-driven sequence comprising host-to-device transfer, CUDA execution, device-to-host transfer, and the synchronization required to complete those operations.

The CPU intervals are classified as transformation measurements. They include the measured encryption call, with output allocation performed by the selected CPU library path, but exclude cipher construction and other application initialization operations.

The GPU resident interval is a device-execution measurement. Input and output buffers are prepared outside the timed kernel region. The measured interval therefore characterizes the CUDA transformation independently of data movement to and from the host.

The GPU pipeline uses prepared pinned host buffers and preallocated device buffers. Its measured sequence includes the synchronous host-to-device (H2D) copy, kernel invocation and completion, device-to-host (D2H) copy, and associated synchronization.

Pinned host memory can facilitate efficient transfers between host and device, although its use does not itself imply overlap between transfers and computation [17, 18].

The measured GPU pipeline is synchronous and does not implement a separate overlapping, multi-stream transfer/compute pipeline.

Importantly, the prepared-buffer boundary excludes the allocation and pinning of host buffers, device-memory allocation, the initial host-side staging of the input, CUDA compilation, and cryptographic key preparation.

It also excludes external application I/O, file-system operations, authentication, and service-level orchestration.

For this reason, the term *pipeline* is preferred to an unqualified application-level *end-to-end* description. Where end-to-end terminology is retained for consistency with the benchmark implementation, it refers strictly to the documented prepared-buffer boundary.

These execution scopes are intentionally different. The CPU-to-GPU speedup associated with the resident measurement is therefore a compute-path comparison,

whereas pipeline speedup is a comparison with the broader, transfer-inclusive GPU processing sequence.

Neither comparison automatically establishes the performance of a complete encryption application.

The archived results also contain selected H2D and D2H host timing intervals. Because these intervals include synchronization effects, they should not be interpreted as isolated PCIe transmission times.

Similarly, subtraction of independently measured GPU resident time from pipeline time produces, at most, an approximate residual-overhead description. It is not an independent direct measurement of pure transfer latency.

This distinction is particularly important when interpreting systems for which resident CUDA throughput substantially exceeds the corresponding prepared-buffer pipeline throughput.

3.5 Timing and Throughput Calculation

CPU execution is timed using Python's monotonic high-resolution `time.perf_counter_ns()` interface.

This timer records elapsed wall-clock intervals in integer nanosecond units. Its numerical representation does not imply that the effective measurement accuracy is one nanosecond.

GPU resident execution is measured using CUDA event timing with the synchronization necessary to obtain completed device-side elapsed time.

CUDA events are appropriate for this purpose because device operations can execute asynchronously with respect to the host process [18, 20].

The prepared-buffer GPU pipeline is measured using host-observed wall-clock timing around the defined synchronous processing sequence.

Consequently, the GPU resident and pipeline measurements use different timing mechanisms as well as different execution boundaries. Their values are not interchangeable observations of the same interval.

For an input containing N processed bytes and an elapsed time T expressed in seconds, throughput is defined as:

$$R = \frac{N}{10^6 T}. \tag{5}$$

The resulting throughput is expressed in decimal megabytes per second (MB/s).

Input sizes are expressed using binary units such as KiB and MiB. This avoids ambiguity between the byte count used in the throughput calculation and

the displayed workload size.

For a fixed workload, the CPU-to-GPU speedup is:

$$S_{\mathrm{CPU/GPU}} = \frac{T_{\mathrm{CPU}}}{T_{\mathrm{GPU}}}. \tag{6}$$

A value of $S_{\mathrm{CPU/GPU}} > 1$ indicates that the selected GPU path completes its measured operation faster than the corresponding CPU reference.

A value below one indicates an advantage for the CPU reference.

Every reported speedup is identified by the CPU backend, CUDA implementation, cryptographic mode, workload size, platform, and GPU timing scope.

For small workloads, latency is also important because fixed costs associated with dispatch, synchronization, and transfer initiation can dominate the measured interval. Throughput-only interpretation can obscure these effects, particularly near the lower end of the workload-size range.

3.6 Experimental Sessions and Statistical Aggregation

Each platform contributes five study sessions identified by the documented seeds. Within each session, twenty timing repetitions are recorded per measured workload configuration.

The session, rather than an individual timing repetition, constitutes the primary experimental replication unit.

Let $T_{s,r}$ denote the measured elapsed time for repetition r within session s.

The session-level median elapsed time is:

$$\widetilde{T}_s = \mathrm{median}_r(T_{s,r}). \tag{7}$$

The principal aggregated elapsed-time statistic is the median of the five session medians:

$$\widetilde{T}_{\mathrm{study}} = \mathrm{median}_{s=1,\ldots,5}\left(\widetilde{T}_s\right). \tag{8}$$

The study's throughput estimator is computed independently from the recorded throughput observations.

For each session, the median of the recorded throughput values is first calculated:

$$\widetilde{R}_s = \mathrm{median}_r(R_{s,r}). \tag{9}$$

The principal throughput statistic is then:

$$\widetilde{R}_{\text{study}} = \text{median}_{s=1,\ldots,5}\left(\widetilde{R}_s\right). \tag{10}$$

This definition preserves the throughput estimator used by the archived analysis instead of deriving every reported rate from an aggregated latency value.

For paired CPU–GPU comparisons, the CPU and GPU timing medians are first obtained within the same session, mode, and workload configuration.

The session-level paired speedup is:

$$S_s = \frac{\widetilde{T}_{\text{CPU},s}}{\widetilde{T}_{\text{GPU},s}}. \tag{11}$$

The reported study-level speedup is:

$$\widetilde{S}_{\text{study}} = \text{median}_{s=1,\ldots,5}(S_s). \tag{12}$$

This estimator is not generally identical to the ratio of independently aggregated CPU and GPU throughput statistics, nor necessarily to the ratio of their independently aggregated latency medians.

The session-level formulation preserves the intended CPU–GPU pairing.

Measurements from different platforms are aggregated separately. CPU references are not pooled between hosts, because the systems use different processors, software environments, and execution conditions.

Session variability is characterized descriptively using the observed range, interquartile range (IQR), and sample coefficient of variation (CV), where meaningful.

For session-level estimates x_s, the sample CV is:

$$\text{CV} = \frac{\sqrt{\frac{1}{n-1}\sum_{s=1}^{n}(x_s - \bar{x})^2}}{\bar{x}}, \qquad n = 5. \tag{13}$$

The number of sessions supporting a directional comparison is also reported when relevant.

For example, a count of $5/5$ means that all five observed session-level speedup values exceed one. It is a descriptive consistency measure, not a hypothesis test or a guarantee that the same direction will hold in future executions.

The analysis does not pool twenty repetitions from five sessions into one set of one hundred independent replications. No observations are discarded solely because they appear extreme.

Confidence intervals and statistical significance claims are not inferred from descriptive ranges or session-level direction counts.

This treatment follows established recommendations concerning experimental variability, reproducibility, and the interpretation of computer-system performance measurements [22–24].

3.7 Experimental Artifacts and Traceability

The benchmarking workflow preserves the evidence required to associate each reported performance result with the original experimental execution.

The principal artifact categories include:

1. **Raw observations.** Individual timing and throughput records associated with the measured configurations and repetitions.
2. **Session summaries.** Derived statistics for each session and workload configuration.
3. **Validation records.** Documented correctness, compilation, and backend verification outcomes.
4. **Environment and configuration metadata.** Hardware descriptions, software dependencies, execution arguments, random seeds, and relevant runtime parameters.
5. **Source-code snapshots.** Archived benchmark and kernel source files permitting source-level comparison between experimental runs.
6. **Cross-session aggregate records.** Session variability, paired speedups, aggregate summaries, and completeness-verification results.

The final analysis uses one selected complete five-session heterogeneous study for each of the three platforms.

These studies comprise fifteen validated sessions and 18,000 raw timing observations.

An independent offline analysis reconstructed the saved session statistics and aggregate estimators from the original raw observations.

The resulting comparison reported no numerical discrepancies within the declared floating-point comparison tolerances. The analysis also verified the presence of the required sessions, workload groups, validation records, and associated source snapshots.

The active benchmark and CUDA source snapshots were compared using SHA-256 hashes. Their matching content supports source-level comparability across the three selected studies.

However, matching source-code hashes do not imply identical compiled binaries, dependencies, driver behavior, or hardware execution.

Similarly, successful artifact verification establishes internal consistency and traceability rather than immunity from every possible measurement or system-level error.

The source archives and analysis outputs retain their original platform and session identities. Historical, pilot, and smoke results are not merged into the principal three-platform estimates.

The integrity assessment therefore connects each principal finding to documented configurations, original run identifiers, and archived observations, permitting subsequent independent inspection and reanalysis.

3.8 Methodological Scope and Limitations

The experimental framework supports controlled interpretation of CPU–GPU performance within the tested configurations. It does not establish universal architectural rankings or application-level encryption performance.

Several limitations are relevant.

First, the three platforms differ simultaneously in CPU model, GPU model, memory system, operating system, driver stack, CUDA runtime, and supporting software libraries.

Consequently, observed differences cannot be causally attributed to Windows versus Linux, consumer versus professional GPUs, or individual library versions.

Second, only one machine or cloud allocation is evaluated per platform, with five study sessions per configuration.

The results provide evidence of within-platform repeatability under the observed conditions but cannot establish population-wide performance characteristics.

Third, the CPU reference measurements characterize the selected PyCryptodome execution paths. The archived records support the intended hardware-assisted AES-NI initialization, but they do not constitute a direct processor instruction trace.

The CPU measurements are not benchmarks of every available AES library or a comprehensive search for the fastest possible multithreaded CPU implementation.

Fourth, the CUDA baseline and optimized-v1 kernels are implementation

candidates rather than representative upper bounds on GPU AES performance.

The study therefore examines the measured implementations and should not be interpreted as a comparison with every published GPU AES optimization technique.

Fifth, the measurement scopes differ deliberately. Resident execution excludes host–device transfers, while the prepared-buffer pipeline includes the selected synchronous transfer and execution operations but excludes buffer preparation and complete application I/O.

These boundaries are essential to interpretation and prevent the resulting throughput figures from being treated as equivalent full-system measurements.

Sixth, the tested workload sizes are discrete and executed in a fixed order. Although a local throughput reduction is observed around 16 MiB in some configurations, the study does not establish its underlying cause or a universal performance threshold.

Driver scheduling, memory effects, clock behavior, thermal conditions, and other environmental mechanisms remain possible influences rather than experimentally isolated explanations.

Seventh, the study is restricted to AES-128 encryption in ECB and CTR modes for the primary heterogeneous comparison. It does not establish performance for every AES key length, authenticated-encryption mode, asynchronous GPU pipeline, or complete storage and network application.

Finally, the statistical analysis emphasizes session-level descriptive consistency. The observed speedup ranges and directional counts do not independently demonstrate statistical significance or guarantee deployment-level performance.

Within these limits, the methodology permits a reproducible comparison of CPU software execution, CPU AES-NI, GPU resident execution, and the tested prepared-buffer GPU pipeline.

Its central contribution is the explicit separation of cryptographic correctness, measured execution scope, statistical aggregation, and practical performance interpretation.

4 Results

This section presents the heterogeneous AES-128 findings obtained from the cryptographic benchmarking framework described in Section 3.

The principal study comprises fifteen validated sessions, five per platform, and 18,000 raw timing observations. It evaluates software CPU AES, hardware-accelerated AES-NI, and baseline and optimized-v1 CUDA implementations using

AES-128 in ECB and CTR modes.

The analysis emphasizes three questions: whether GPU execution provides an advantage over the selected CPU references; whether that advantage persists when transfers are included; and how the observed relationships vary across workload sizes, cipher modes, and computing configurations.

Unless otherwise stated, input sizes use binary units (KiB and MiB), while throughput is reported in decimal megabytes per second (MB/s). The principal heterogeneous results are medians of five session-level estimates. Paired speedups are medians of corresponding session-level CPU-to-GPU timing ratios, not ratios of independently aggregated throughput values.

4.1 Three-Platform AES-128 CPU–GPU Evaluation

The principal experiment evaluates AES-128 encryption across three heterogeneous computing configurations:

1. Windows 10 with an Intel Core i5-10300H and GeForce GTX 1650 Ti;
2. Windows 11 with an Intel Core i7-11850H and T600 Laptop GPU;
3. Google Colab/Linux with an exposed Intel Xeon host CPU and Tesla T4 GPU.

The GTX 1650 Ti represents a consumer laptop GPU, while the T600 is an entry-level professional mobile GPU. Both have 4 GB of device memory. The T4 provides a datacenter-oriented comparison.

The CUDA baseline assigns an independent AES block to each CUDA thread, with a launch configuration of 128 threads per CUDA block. The optimized-v1 kernel is evaluated as a separate lightweight implementation candidate.

Both kernels are measured in ECB and CTR modes. Their results are interpreted as implementation measurements, not limits on the attainable performance of GPU AES.

The two GPU timing scopes are evaluated separately:

1. **Resident:** device-resident CUDA execution excluding host–device data transfers.
2. **Pipeline:** the synchronous, prepared-buffer sequence including H2D transfer, kernel execution, D2H transfer, and required synchronization.

The pipeline scope excludes buffer allocation, initial host staging, cryptographic initialization, and external application I/O. It therefore does not represent full application-level encryption.

The three selected studies contain five sessions per platform and cover 1 KiB, 16 KiB, 1 MiB, 16 MiB, and 32 MiB.

The independent audit confirmed completeness of the selected datasets and reproduced the saved statistical estimators without detected numerical discrepancies at the specified floating-point tolerances.

The following analysis uses the archived three-platform study results exclusively.

4.1.1 GPU Pipeline Performance Across Input Sizes

The first practical comparison considers the throughput of the prepared-buffer GPU pipeline, which includes the measured host–device transfers.

Figure 2 presents the resulting throughput across all five workload sizes for both CUDA implementations and both AES modes.

For 32 MiB CTR workloads, baseline pipeline throughput reaches approximately 1188.9 MB/s on the GTX 1650 Ti, 2049.4 MB/s on the T600 Laptop GPU, and 3713.9 MB/s on the T4.

The corresponding resident throughputs are 4585.6, 4328.9, and 9166.8 MB/s, respectively.

These results demonstrate substantial differences between isolated GPU execution and transfer-inclusive processing.

They also show that the T4 achieves the highest absolute GPU throughput in this matched large-buffer CTR configuration.

However, the comparison includes differences in the complete host systems, drivers, CUDA environments, and device architectures. It cannot establish that GPU product category alone causes the observed ordering.

The two 4 GB laptop GPUs remain practically relevant because they deliver substantial absolute throughput on the selected workloads without requiring a datacenter GPU.

Whether that throughput constitutes useful acceleration depends on the CPU reference and the scope of the comparison.

4.1.2 CPU AES-NI Versus GPU Pipeline Speedup

The central offloading question is whether the GPU pipeline completes the measured operation faster than the corresponding hardware-accelerated CPU AES-NI path.

Figure 3 presents the paired pipeline speedup relative to each platform's own

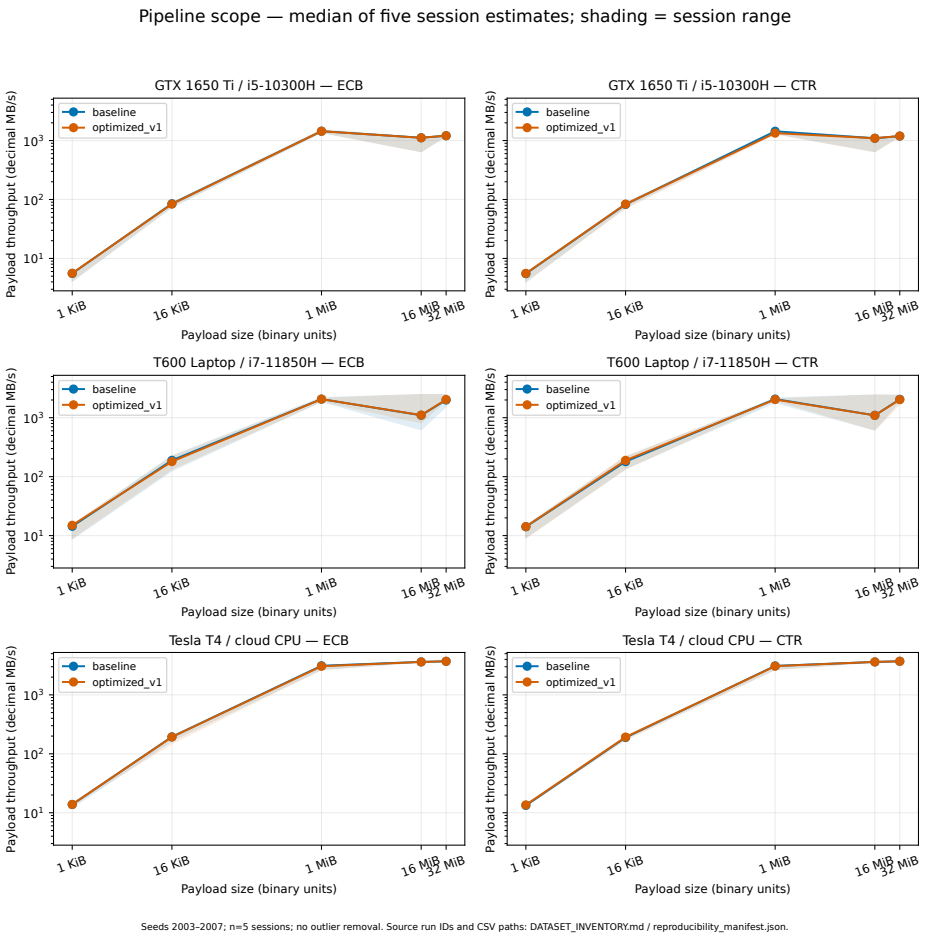


Fig. 2 — Prepared-buffer GPU pipeline throughput for AES-128 ECB and CTR on the three platforms. Values are medians of five session-level throughput medians, expressed in decimal MB/s. Input sizes are binary quantities. Observed session ranges describe measured variability and are not confidence intervals. The three panels represent distinct systems, rather than a controlled GPU-class comparison.

AES-NI reference.

The results are strongly configuration-dependent.

On the consumer GTX 1650 Ti, baseline CTR at 32 MiB achieves a pipeline speedup of 0.935× relative to AES-NI. The five-session range is 0.918–1.008×, with only one session above parity.

The corresponding CUDA v1 result is 0.954×, also with a gain in only one of five sessions.

Thus, neither implementation demonstrates a consistently faster transfer-inclusive pipeline than AES-NI on the GTX platform.

This finding contrasts with the same GPU's comparison against the selected software CPU AES implementation.

At 32 MiB CTR, baseline pipeline execution is approximately 4.566× faster than software CPU AES, with an advantage in all five sessions.

The difference between these conclusions is not contradictory: the CPU software and AES-NI execution paths have substantially different performance.

The T600 Laptop GPU provides a particularly informative positive case.

For baseline CUDA in CTR mode at 32 MiB, the pipeline achieves a median paired speedup of

$$\widetilde{S}_{\mathrm{T600,CTR,P}} = 1.353841. \tag{14}$$

The observed session range is 1.128769–1.391103, and all five sessions exceed parity.

This corresponds to an approximately 35.38% improvement in the reported CPU-to-GPU speedup ratio, or an approximately 26.14% reduction in the median paired latency ratio.

The observed speedup IQR is 0.030896, with a sample coefficient of variation of approximately 8.18%.

This result is the strongest all-five-session pipeline advantage over AES-NI among the tested laptop conditions.

Nevertheless, it is specific to the T600 host configuration, baseline CUDA implementation, CTR mode, 32 MiB input size, and the documented prepared-buffer measurement boundary.

It does not establish a guaranteed advantage for every 4 GB GPU, every Windows laptop, or every AES workload.

The T600 baseline and optimized-v1 CTR pipelines also outperform AES-NI in all five sessions at 1 MiB. At 16 MiB, however, the direction is inconsistent,

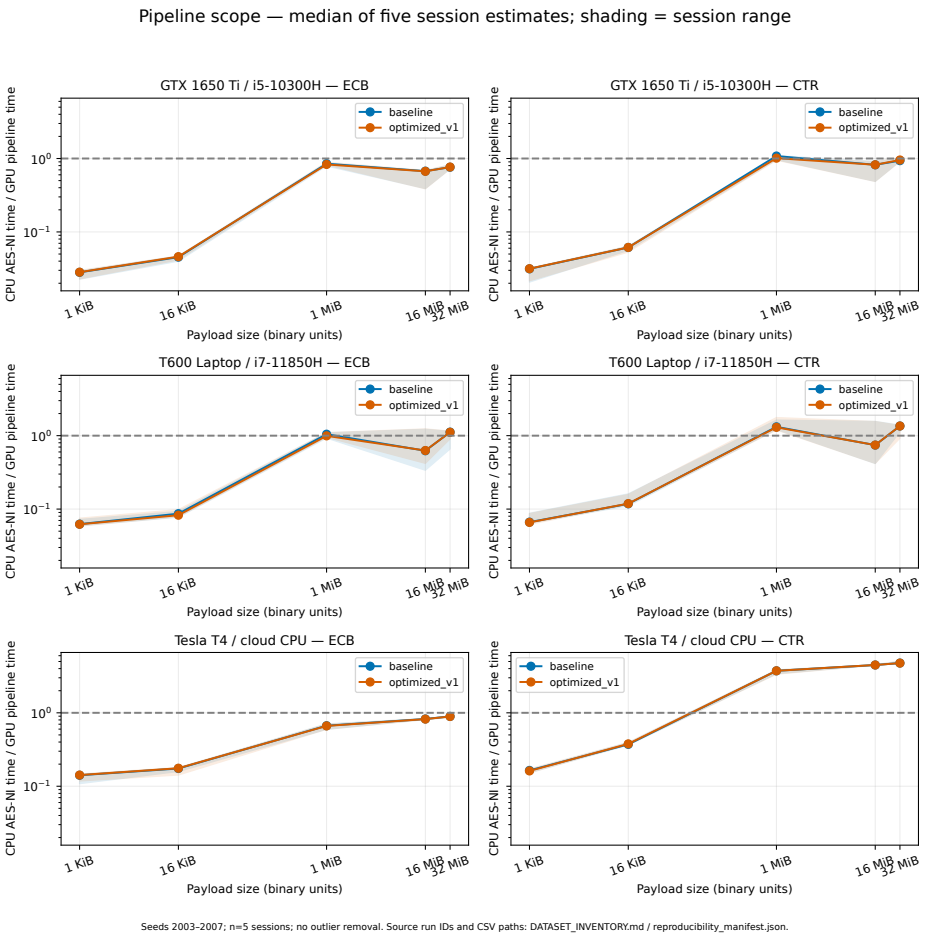


Fig. 3 — Paired AES-128 GPU pipeline speedup relative to the corresponding host CPU AES-NI transformation. Speedup is the median of five session-specific CPU/GPU median-time ratios. The reference value of one denotes parity. Values above one indicate a shorter measured GPU pipeline interval. Session ranges are descriptive, and each platform has its own CPU reference.

illustrating the importance of examining the complete size grid rather than a single favorable operating point.

On the Tesla T4, baseline CTR pipeline execution reaches approximately 4.761× the paired host AES-NI speed at 32 MiB, with an advantage in all five sessions.

In the same 32 MiB ECB workload, the T4 baseline pipeline achieves only approximately 0.893× the host AES-NI reference and loses in every session.

Therefore, even the datacenter GPU configuration does not provide a universal pipeline advantage over hardware-accelerated CPU AES.

The mode-dependent CPU reference is essential to interpreting this contrast.

4.1.3 Resident Versus Pipeline Measurements

The distinction between resident-only and transfer-inclusive execution is the principal methodological finding of the heterogeneous study.

Table 2 reports the paired GPU speedup relative to CPU AES-NI at 32 MiB.

The table includes both CUDA implementations and both AES modes on all three platforms, retaining positive, negative, and mixed-session outcomes.

Table 2
AES-128 speedup relative to paired CPU AES-NI for 32 MiB inputs. R denotes GPU resident execution and P the prepared-buffer pipeline. Values are medians of five paired session-level timing ratios. The last column gives the number of sessions with pipeline speedup above one.

Platform	CUDA	Mode	R	P	P > 1
GTX 1650 Ti	Base	ECB	3.091	0.761	0/5
GTX 1650 Ti	Base	CTR	3.693	0.935	1/5
GTX 1650 Ti	v1	ECB	3.082	0.765	0/5
GTX 1650 Ti	v1	CTR	3.698	0.954	1/5
T600	Base	ECB	2.215	1.118	3/5
T600	Base	CTR	2.666	1.354	5/5
T600	v1	ECB	2.158	1.112	4/5
T600	v1	CTR	2.541	1.354	4/5
T4	Base	ECB	2.207	0.893	0/5
T4	Base	CTR	11.758	4.761	5/5
T4	v1	ECB	2.202	0.891	0/5
T4	v1	CTR	11.838	4.756	5/5

Every 32 MiB resident comparison reported in Table 2 exceeds the paired AES-NI reference in all five sessions.

However, the same consistency does not hold for the broader GPU pipeline.

On the GTX 1650 Ti, every 32 MiB pipeline median remains below parity. The ECB pipeline loses in all five sessions for both CUDA implementations.

On the T600, the CTR baseline retains a consistent pipeline advantage, whereas ECB and optimized-v1 CTR show mixed session-level outcomes.

On the T4, CTR retains a substantial pipeline advantage, while ECB consistently favors the CPU AES-NI reference.

These findings demonstrate that a resident-only GPU result cannot be used as a substitute for measuring the corresponding transfer-inclusive processing path.

The same CUDA kernel can appear substantially faster than the CPU in resident measurements while the complete measured pipeline produces a smaller advantage, no consistent advantage, or a reversal in relative ranking.

The practical implication is not that GPU acceleration is inherently ineffective.

Instead, data residency, workload size, execution scope, and the performance of the CPU reference determine whether offloading is advantageous under the tested processing sequence.

The distinction is consistent with the broader importance of data movement in heterogeneous computing [14].

4.1.4 Pipeline Throughput Retention

A complementary perspective examines how much of the device-resident throughput is retained when the measured GPU processing boundary is expanded to include data movement and synchronization.

Figure 4 presents the pipeline-to-resident throughput ratios.

At 32 MiB, the pipeline generally retains approximately 25–26% of resident throughput on the GTX 1650 Ti, 46–48% on the T600, and about 40% on the T4.

These differences are relevant because the GPU kernels process the same number of bytes in both measurement scopes, while the pipeline includes additional operations.

Nevertheless, the measured ratios cannot be interpreted as the pure fraction of time spent on PCIe transfers.

The resident kernel is measured using CUDA events, whereas pipeline timing uses the host-observed synchronous processing interval.

The archived H2D and D2H component intervals also include synchronization effects.

For the T4 32 MiB workload, representative recorded H2D and D2H intervals

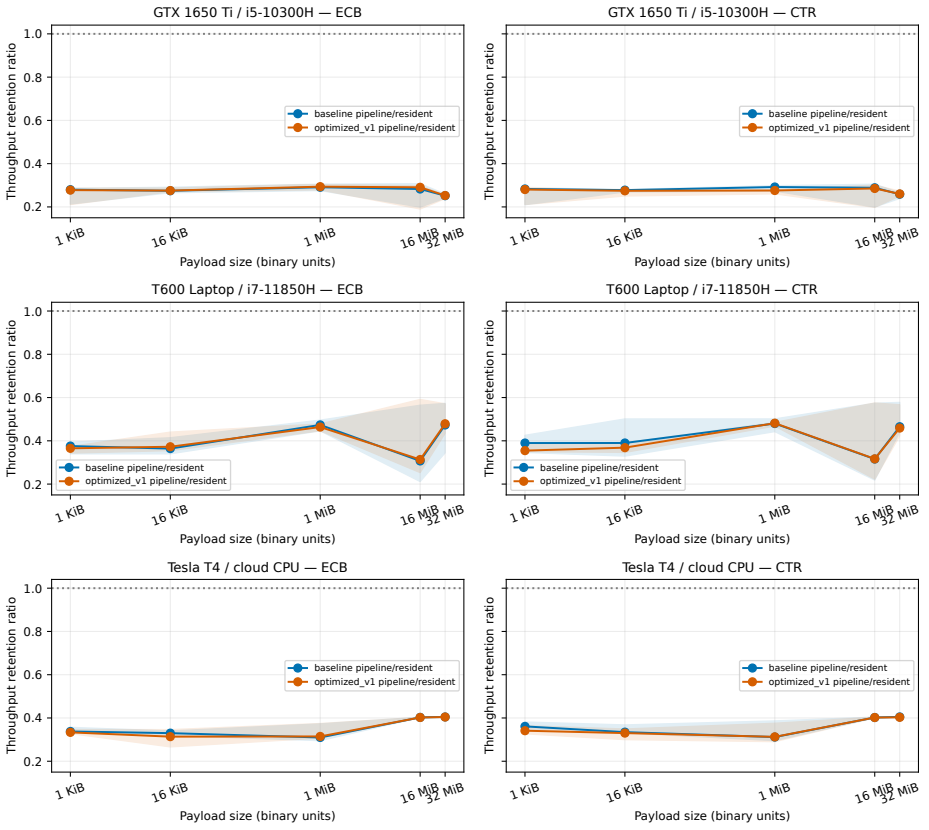


Fig. 4 — Ratio of prepared-buffer GPU pipeline throughput to GPU resident throughput across the tested configurations. The ratios describe throughput retention under different timing boundaries; they are not direct measurements of PCIe transfer efficiency or the fraction of elapsed time spent copying data.

are approximately 2.77 and 2.60 ms, respectively.

These values describe the instrumented component intervals and do not independently establish physical PCIe transfer times.

Accordingly, the results support a measurement-scope interpretation without claiming a complete causal decomposition of pipeline overhead.

4.1.5 Cipher-Mode Dependence and the T4 Contrast

The T4 measurements reveal a particularly important difference between absolute GPU performance and speedup relative to a host CPU.

At 32 MiB, baseline T4 GPU pipeline throughput is approximately 3713 MB/s for ECB and 3714 MB/s for CTR.

The corresponding resident throughputs are approximately 9177 and 9167 MB/s.

Thus, the GPU's measured throughput is nearly identical for the two cipher modes under these conditions.

However, the relative CPU–GPU pipeline speedups are markedly different: approximately 0.893× for ECB and 4.761× for CTR.

The difference is associated with the corresponding host CPU performance.

At 32 MiB, the T4 host's AES-NI ECB throughput is approximately 5.378× its AES-NI CTR throughput, with a session range of 5.250–5.795×.

The two Windows systems show substantially smaller ECB-to-CTR AES-NI ratios of approximately 1.26× at the same input size.

The T4 disparity is reproducible across the five observed sessions, but its underlying cause was not isolated.

Potential influences include the host library's mode-specific execution, counter processing, processor behavior, and cloud runtime conditions.

The results do not establish that any particular library version, virtualization mechanism, or CPU microarchitecture feature causes the disparity.

This finding illustrates a broader interpretive problem: a high CPU-to-GPU speedup can arise from a slower CPU reference as well as from faster GPU execution.

For this reason, the study reports both absolute throughput and speedup relative to explicitly identified CPU paths.

The T4's large CTR speedup must not be interpreted as a 4.761× absolute GPU throughput advantage over the laptop GPUs, because those speedups use different host CPUs.

4.1.6 Workload-Size Effects and Session Variability

The measured throughput does not always increase monotonically with input size.

A local throughput reduction is observed around 16 MiB on the GTX 1650 Ti.

Across the two CUDA implementations, two modes, and two timing scopes, all eight GTX groups show a 16 MiB local dip below both the 1 MiB and 32 MiB measurements in all five study sessions.

On the T600, the corresponding joint local-dip counts vary from three to five sessions, depending on the implementation, cipher mode, and timing scope.

On the T4, resident measurements exhibit the local dip in four sessions, whereas the pipeline measurements do not show the same joint local-dip pattern.

Indeed, T4 pipeline throughput increases from 1 MiB to 16 MiB in every session.

The 16 MiB feature is therefore not universal across the tested GPUs or timing boundaries.

The fixed ascending workload-size order also prevents the experiment from separating buffer-size effects completely from time-dependent system behavior.

No causal explanation is assigned to the observed reduction.

In particular, the data do not establish that WDDM scheduling, thermal throttling, GPU memory behavior, or PCIe transfers are responsible for the feature.

Between-session variability is summarized in Figure 5.

The T600 environment exhibits appreciable variation for selected pipeline configurations, while the GTX and T4 show different variability profiles.

These observations reinforce the importance of preserving session-level statistics and not reporting a single maximum-throughput result as representative of the complete study.

In addition to aggregate medians, the analysis retains observed session ranges, interquartile ranges, coefficients of variation, and counts of sessions supporting a particular performance direction.

A $5/5$ directional result indicates consistency among the five observed sessions. It does not constitute a formal significance test or a guarantee of the same outcome on another device.

4.1.7 Baseline Versus Optimized-v1 CUDA

The optimized-v1 implementation does not provide a uniform performance advantage over the CUDA baseline.

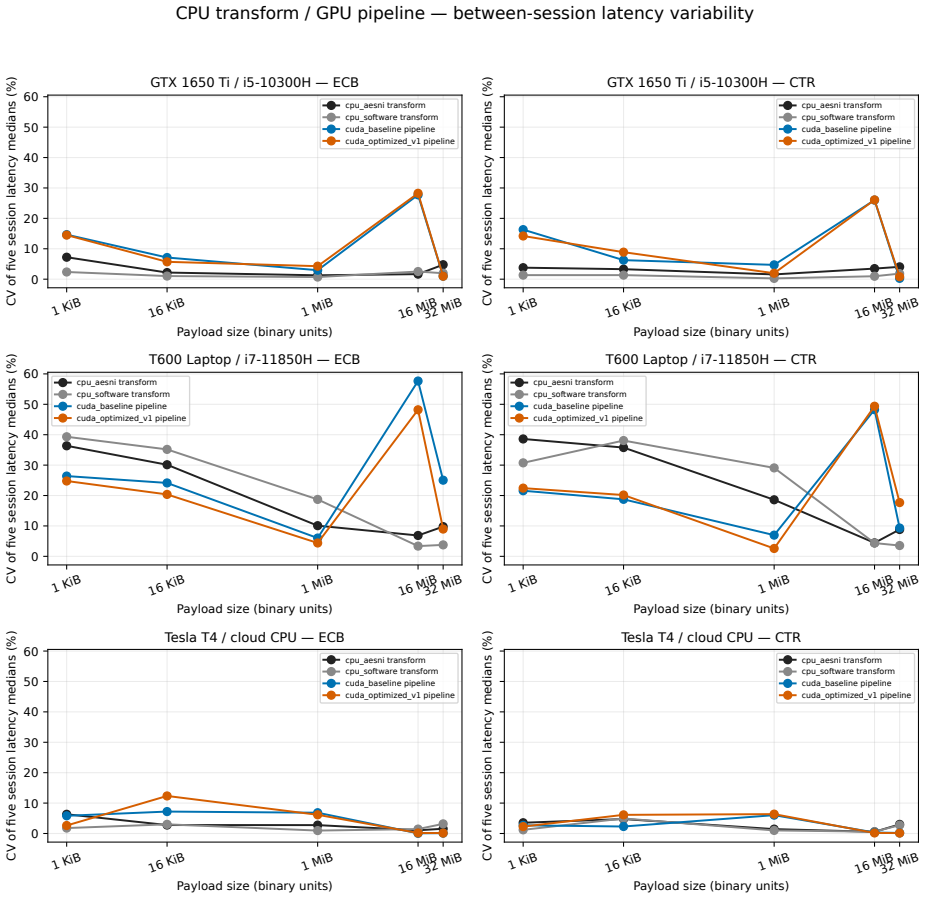


Fig. 5 — Between-session variability in CPU transformation and GPU pipeline latency. The coefficient of variation is computed from the five session-level median times using sample standard deviation divided by the session mean. The figure describes observed variability rather than a population confidence interval.

Its relative behavior changes with GPU model, AES mode, workload size, and measurement scope.

For example, at 32 MiB on the T4, optimized-v1 resident CTR shows a small advantage over baseline, while resident ECB is slightly slower.

On the GTX 1650 Ti, the optimized-v1 pipeline does not consistently remove the disadvantage relative to AES-NI.

On the T600, the optimized-v1 CTR pipeline has a median speedup above one at 32 MiB, but the direction reverses in at least one session.

The observed differences are therefore insufficient to claim that the lightweight optimization candidate is generally superior.

The result is important methodologically because kernel changes that improve a particular resident measurement do not necessarily improve the complete measured processing pipeline.

The main contribution of the CUDA comparison is consequently the controlled assessment of implementation and timing-scope dependence, rather than a new optimized AES performance record.

5 Discussion

The findings of this study extend beyond a comparison of encryption throughput. They provide empirical evidence that the benefit of GPU-based AES execution depends on the host processor, cipher mode, workload size, data location, and the operations included within the measurement boundary. These factors are particularly relevant when cryptographic acceleration is considered for accessible computing systems, communication infrastructure, and heterogeneous data-processing platforms.

The principal result is twofold. First, even relatively accessible 4 GB NVIDIA laptop GPUs can deliver substantial AES-128 acceleration under selected conditions. Second, higher device-resident throughput does not guarantee that transferring data to and from the GPU will remain advantageous over a CPU already equipped with AES-NI instructions.

The three-platform evaluation therefore supports neither a universal preference for CPU execution nor a general recommendation to offload AES to the GPU. Instead, it identifies measured conditions under which acceleration is beneficial and provides a reproducible procedure for assessing whether those conditions apply to a particular processing system.

This distinction is important for both technical specialists and system architects. The relevant deployment question is not simply which processor encrypts data faster, but which execution path completes the required cryptographic op-

eration at an acceptable cost in throughput, latency, resource usage, and data movement.

5.1 When Can GPU Encryption Outperform AES-NI?

The experimental results provide direct evidence that GPU execution can outperform the selected hardware-accelerated CPU AES-NI path, including when host–device transfers are incorporated into the measured processing sequence.

The clearest example among the two tested 4 GB laptop GPUs is the NVIDIA T600 Laptop GPU.

For AES-128 CTR encryption of a 32 MiB buffer, the baseline CUDA prepared-buffer pipeline achieves a median paired speedup of 1.354× relative to the corresponding CPU AES-NI transformation on the Intel Core i7-11850H host.

The observed speedup ranges from 1.129× to 1.391× across five study sessions, with the GPU advantage present in all five.

This represents approximately 26% lower paired processing latency for the defined benchmark operation, based on the reported median speedup ratio.

The result demonstrates that GPU offloading can remain advantageous even when the measured workload includes synchronous host-to-device transfer, CUDA execution, device-to-host transfer, and synchronization.

However, the result does not establish that the T600 would outperform every optimized CPU implementation or that the same advantage would persist in a complete telecommunications application.

The consumer GeForce GTX 1650 Ti presents an equally important counterexample.

At 32 MiB in CTR mode, its baseline CUDA pipeline achieves a 4.566× median speedup over the selected software CPU AES implementation, but only 0.935× relative to its host's AES-NI implementation.

The GPU therefore provides substantial acceleration compared with software-only AES without demonstrating a consistent advantage over the stronger hardware-assisted CPU reference.

The NVIDIA Tesla T4 provides a further contrast.

For the 32 MiB CTR workload, its baseline CUDA pipeline achieves approximately 4.761× speedup relative to the paired host AES-NI implementation, with an advantage observed in all five sessions.

For ECB mode under otherwise matching conditions, however, the T4 pipeline achieves only 0.893× the host AES-NI reference.

This large difference in relative speedup does not correspond to an equally

large difference in GPU encryption throughput. At 32 MiB, T4 baseline pipeline throughput is approximately 3.71 GB/s for both ECB and CTR.

Instead, the host AES-NI execution paths exhibit markedly different mode-dependent throughput. The measured ECB-to-CTR AES-NI throughput ratio on the T4 host is approximately 5.38, compared with approximately 1.26 on the two Windows hosts.

The underlying cause of the T4 CPU disparity was not isolated. Therefore, it would be inappropriate to attribute the observed CTR advantage exclusively to GPU hardware, CUDA execution efficiency, or a particular software-library version.

Taken together, the results establish a qualified but practically meaningful conclusion: GPU AES execution can outperform a hardware-accelerated CPU reference, including in a transfer-inclusive pipeline, but only under specific combinations of hardware, workload, mode, and implementation.

The distinction between absolute throughput and host-relative speedup is essential when selecting computing resources.

5.2 Why Parallel Execution and Data Location Matter

AES-128 CTR is well suited to parallel execution because the input blocks used to generate the keystream can be processed independently, provided the counter values are assigned correctly and are not reused with the same key.

A CUDA implementation can distribute these independent block computations across many GPU threads.

This execution structure offers an architectural reason why GPUs may provide substantial throughput for large collections of independent blocks. Earlier research has also investigated AES acceleration on CUDA devices [11].

However, the experiments do not isolate GPU parallelism as the sole cause of the observed speedup. The measured result reflects the interaction between the CUDA implementation, host CPU, memory behavior, runtime environment, and timing boundary.

Modern CPUs provide an alternative form of acceleration through dedicated cryptographic instructions.

AES-NI reduces the cost of selected AES operations by executing cryptographic transformations through processor-supported instructions. Consequently, a software-only AES implementation is not an adequate standalone baseline for deployment decisions on AES-NI-capable processors.

This principle is consistent with earlier work showing how comparisons between optimized CPU and GPU implementations depend on the selected imple-

mentations and their execution conditions [15].

The present study demonstrates this effect directly. A GPU advantage measured against software AES can become substantially smaller or disappear when the CPU reference uses AES-NI.

Data location introduces another important distinction.

For the 32 MiB workloads, the GPU resident paths outperform the paired AES-NI CPU transformation on all three systems, for both CUDA implementations and both evaluated AES modes, in all five sessions.

Nevertheless, the corresponding prepared-buffer pipeline measurements are less favorable.

On the GTX 1650 Ti, the pipeline retains approximately one quarter of resident throughput at 32 MiB. The corresponding proportions are approximately 46–48% on the T600 and about 40% on the T4.

These values do not measure pure PCIe transfer efficiency. They express the reduction in observed throughput between different timing boundaries, including the additional host-driven operations represented by the synchronous pipeline.

This interpretation is consistent with the findings of Gregg and Hazelwood [14], who emphasized that the location of application data and the cost of movement are essential to meaningful CPU–GPU comparisons.

In practical terms, a system whose data are already resident on the GPU may avoid some transfers that are necessary in a host-memory-based workflow.

Conversely, a system that receives every input buffer in CPU memory and requires every encrypted output to return immediately to the CPU may obtain less benefit from the accelerator.

The results therefore support data placement as a central architectural consideration, although the potential benefits of persistent GPU residency in complete applications remain to be measured.

5.3 Implications for Telecommunications Systems

The present measurements do not establish a consistent GPU pipeline advantage for the smallest workloads. Thus, they do not justify routing every small network packet through a discrete GPU.

The measured 32 MiB results demonstrate that substantial processing volumes can favor the GPU under particular configurations. The T600 result shows that this advantage is not restricted to datacenter-class GPUs.

Nevertheless, a single contiguous 32 MiB benchmark buffer is not equivalent to a collection of independently authenticated network packets.

Batch preparation, packet metadata, cryptographic state, nonce assignment,

authentication, output delivery, and queueing delays must be accounted for before extrapolating to a real communication system.

The 32 MiB results should therefore be interpreted as evidence that large aggregate processing workloads warrant GPU evaluation, not as proof that individual telecommunication packets will be encrypted faster on the GPU.

5.4 Authenticated Encryption and Practical Security

The heterogeneous performance study focuses on AES-128 ECB and CTR because these modes permit controlled evaluation of independent block-level computation.

This scope is useful for benchmarking, but it does not encompass all requirements of secure communication.

In particular, ECB does not provide appropriate confidentiality for general structured application data.

CTR provides confidentiality when correctly configured and used, but does not by itself provide message authentication or integrity protection.

Practical network protocols generally require additional cryptographic properties, frequently provided by authenticated-encryption constructions.

AES-GCM combines counter-mode AES processing with authentication based on GHASH [7].

The study does not include a validated GPU AES-GCM pipeline.

Therefore, measured GPU ECB and CTR throughput cannot be treated as AES-GCM throughput, and the T600 or T4 CTR speedups cannot be transferred directly to TLS or IPsec deployments.

A complete AES-GCM GPU implementation would require evaluation of AES counter processing, GHASH, authentication-tag generation and verification, nonce management, and the relevant transfer and application-processing costs.

5.5 Limitations and Threats to Validity

The methodological limitations are detailed in Section 3.8.

The independently reconstructed statistics and archived source evidence strengthen internal traceability, but do not eliminate these external-validity and measurement-scope limitations.

5.6 Future Research Directions

The findings identify several directions for subsequent research, without requiring those extensions to establish the principal contribution of the present study.

A first direction is application-level evaluation of cryptographic offloading in communication systems.

Such research could measure AES-GCM within representative network-processing workloads, including packet batching, authentication, queueing latency, and delivery of processed data.

This would determine whether the advantages observed for large AES-CTR buffers persist when complete security and communication operations are included.

A second direction is adaptive CPU–GPU execution selection.

The observed differences between small inputs, larger workloads, resident computation, and transfer-inclusive processing suggest that heterogeneous schedulers could benefit from workload-aware decisions.

A practical investigation would evaluate throughput and latency together, rather than assuming that the fastest bulk encryption path also minimizes response time for individual packets.

A third direction concerns data-movement-aware system design.

Asynchronous execution, overlapping transfers and computation, device-resident processing, and supported direct-I/O technologies could alter the cost of accelerator offloading.

The present measurements provide a synchronous reference against which such implementations could be assessed, but do not predict their attainable speedup.

A fourth direction is broader validation across independent hardware and software systems.

Additional CPU libraries, multithreaded AES-NI implementations, newer GPUs, and dedicated network cryptographic accelerators would help determine which observations remain stable across architectures.

More densely sampled and randomized workload sizes, combined with synchronized instrumentation, could also help investigate the observed non-monotonic throughput behavior and improve performance prediction.

6 Conclusion

More than two decades after the standardization of the Advanced Encryption Standard (AES), the evolution of computing architectures continues to influence how efficiently symmetric encryption can be implemented and deployed. Building on a retrospective examination of the transition from DES and Triple DES to AES, this study investigated the practical performance of modern CPU and GPU

cryptographic implementations, with particular emphasis on the conditions under which GPU acceleration can outperform CPUs equipped with dedicated AES instructions.

The principal contribution of this work is a validation-first, reproducible benchmarking methodology that integrates cryptographic correctness verification, deterministic and paired workloads, explicitly defined measurement boundaries, session-based statistical aggregation, and preservation of the experimental evidence required for independent analysis. Rather than presenting encryption throughput as an implementation-independent characteristic of AES, the framework distinguishes the performance of software CPU execution, hardware-assisted AES-NI, device-resident CUDA computation, and a synchronous GPU pipeline that includes host–device transfers and synchronization.

The methodology was evaluated across three heterogeneous computing configurations: a Windows 10 system equipped with a consumer NVIDIA GeForce GTX 1650 Ti, a Windows 11 system equipped with an NVIDIA T600 Laptop GPU, and a Linux-based Google Colab environment using an NVIDIA Tesla T4. The two laptop GPUs each provide 4 GB of device memory, demonstrating the relevance of the study to accessible computing hardware as well as datacenter-oriented acceleration.

The experimental campaign comprised fifteen study sessions and 18,000 raw timing observations. Cryptographic validation preceded performance measurement, while an independent analysis reconstructed the saved statistical estimators without detecting numerical discrepancies within the specified tolerances. These procedures provide traceability between the measured implementations, their execution environments, and the reported performance conclusions.

The results demonstrate that GPU acceleration can outperform even a hardware-accelerated CPU AES-NI implementation under selected operating conditions. For AES-128 CTR encryption of a 32 MiB workload, the baseline CUDA implementation on the T600 Laptop GPU achieved a median paired speedup of 1.354× over its host CPU AES-NI execution path, with an advantage observed in all five sessions. This corresponds to approximately 26% lower paired processing latency within the defined prepared-buffer GPU pipeline.

The Tesla T4 achieved a substantially larger 4.761× paired pipeline speedup for the same AES mode and workload size, also outperforming its host AES-NI reference in all five sessions. These findings establish that a transfer-inclusive GPU execution path can provide an advantage over CPU hardware-assisted encryption, including on a comparatively accessible 4 GB laptop GPU.

However, the experiments also demonstrate why this advantage cannot be generalized without considering the measurement boundary and CPU reference.

On the consumer GTX 1650 Ti, the 32 MiB baseline CTR pipeline achieved a 4.566× speedup over conventional software AES but only 0.935× relative to AES-NI. The T4 likewise exhibited a mode-dependent contrast: despite similar absolute GPU throughput for ECB and CTR at 32 MiB, its ECB pipeline achieved only 0.893× the corresponding host AES-NI performance.

Across all three platforms, device-resident CUDA execution outperformed the paired AES-NI reference at 32 MiB for both tested CUDA implementations and both encryption modes, with the advantage observed in all five sessions per configuration. Nevertheless, the same consistency did not extend to transfer-inclusive pipeline execution. The lightweight optimized-v1 CUDA candidate also failed to provide uniform improvements across platforms, workload sizes, cipher modes, and measurement scopes.

Together, these results demonstrate that high isolated kernel throughput is neither a sufficient condition nor a reliable proxy for advantageous GPU offloading. The observed performance relationship depends on the location of data, the costs of host–device movement, the selected CPU implementation, and the processing operations included within the timed interval.

For telecommunications and heterogeneous data-processing systems, the findings have a practical architectural implication. CPU AES-NI remains a strong reference for small, latency-sensitive, or host-resident workloads, whereas GPU processing warrants evaluation for suitable large, parallelizable, or device-resident operations. The observed 32 MiB CTR advantages indicate that GPU acceleration may be valuable for bulk cryptographic processing and appropriately batched workloads, rather than for every individual communication packet.

These observations support workload-aware selection of cryptographic execution paths instead of a universal preference for either CPU or GPU execution. Nevertheless, the present experiments do not benchmark complete TLS, IPsec, or application-level authenticated encryption. Their results therefore provide evidence for architectural assessment rather than a direct deployment recommendation for secure communication protocols.

The principal limitations arise from the evaluation of one system or cloud allocation per platform, five study sessions per configuration, selected library and CUDA implementations, and a synchronous prepared-buffer GPU pipeline. Differences among the tested hardware and software environments prevent causal attribution to operating systems, GPU product classes, or individual runtime components. Furthermore, the tested ECB and CTR workloads do not establish performance for complete authenticated-encryption or network-processing systems.

Future research may extend the methodology to application-level AES-GCM

evaluation, network-packet batching, asynchronous transfer–compute pipelines, workload-aware CPU–GPU scheduling, and newer architectures with dedicated cryptographic acceleration. Such studies could investigate whether the performance advantages observed for large independent AES workloads remain beneficial when authentication, communication latency, resource contention, and complete application data paths are considered.

In conclusion, this study establishes that GPU acceleration is a viable but conditional alternative to hardware-assisted CPU AES execution. Its significance lies not in claiming a universally faster implementation, but in demonstrating reproducible conditions under which GPU offloading succeeds or fails. By combining validated cryptographic execution, explicit measurement boundaries, and independently auditable experimental results, the proposed methodology provides a practical foundation for evaluating cryptographic acceleration and informing the design of future heterogeneous computing and communication systems.

7 Acknowledgments

Placeholder acknowledgments. Replace with appropriate funding or support information, or remove the paragraph if no acknowledgments are needed.

References

[1] National Institute of Standards and Technology. Data encryption standard (des). Federal Information Processing Standards Publication 46-3, National Institute of Standards and Technology, 1999. URL `https://csrc.nist.gov/pubs/fips/46-3/final`.

[2] National Bureau of Standards. Des modes of operation. Federal Information Processing Standards Publication 81, National Bureau of Standards, 1980. URL `https://csrc.nist.gov/pubs/fips/81/final`.

[3] Elaine Barker and Nicky Mouha. Recommendation for the triple data encryption algorithm (tdea) block cipher. NIST Special Publication 800-67 Rev. 2, National Institute of Standards and Technology, 2017.

[4] Elaine Barker and Allen Roginsky. Transitioning the use of cryptographic algorithms and key lengths. NIST Special Publication 800-131A Rev. 2, National Institute of Standards and Technology, 2019.

[5] National Institute of Standards and Technology. Advanced encryption standard (aes). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards and Technology, 2023.

[6] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001.

[7] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.

[8] Intel Corporation. Intel advanced encryption standard instructions (aes-ni), 2012. URL https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html.

[9] Svetlin A. Manavski. Cuda compatible gpu as an efficient hardware accelerator for aes cryptography. In *Proceedings of the IEEE International Conference on Signal Processing and Communications*, 2007. doi: 10.1109/ICSPC.2007.4728256.

[10] Owen Harrison and John Waldron. Aes encryption and decryption on the gpu. In *GPU Gems 3*, chapter 36. Addison-Wesley Professional, 2007. URL https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-36-aes-encryption-and-decryption-gpu.

[11] Keisuke Iwai, Naoki Nishikawa, and Takakazu Kurokawa. Acceleration of aes encryption on cuda gpu. *International Journal of Networking and Computing*, 2(1):131–145, 2012. doi: 10.15803/ijnc.2.1_131.

[12] Canhui Wang and Xiaowen Chu. Gpu accelerated aes algorithm, 2019. URL https://arxiv.org/abs/1902.05234.

[13] Cihangir Tezcan. Optimization of advanced encryption standard on graphics processing units. *IEEE Access*, 9:67315–67326, 2021. doi: 10.1109/ACCESS.2021.3077551.

[14] Chris Gregg and Kim Hazelwood. Where is the data? why you cannot debate cpu vs. gpu performance without the answer. In *2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 134–144, 2011. doi: 10.1109/ISPASS.2011.5762730.

[15] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100x gpu vs. cpu myth: An evaluation of throughput computing on cpu and gpu. In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, pages 451–460, 2010. doi: 10.1145/1815961.1816021.

[16] Wai-Kong Lee, Seog Chung Seo, Hwajeong Seo, Dong Cheon Kim, and Seong Oun Hwang. Speed record of aes-ctr and aes-ecb bit-sliced implementation on gpus. *IEEE Embedded Systems Letters*, 16(4):481–484, 2024. doi: 10.1109/LES.2024.3409725.

[17] NVIDIA Corporation. Cuda c++ programming guide, n.d.. URL https://docs.nvidia.com/cuda/cuda-c-programming-guide/. Accessed 2026.

[18] NVIDIA Corporation. Cuda c++ best practices guide, n.d.. URL https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Accessed 2026.

[19] Mark Harris. How to overlap data transfers in cuda c/c++. NVIDIA Technical Blog, 2012. URL https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/.

[20] CuPy. Performance best practices. https://docs.cupy.dev/en/stable/user_guide/performance.html, n.d.. Accessed 2026.

[21] CuPy. cupy.rawkernel. https://docs.cupy.dev/en/stable/reference/generated/cupy.RawKernel.html, n.d.. Accessed 2026.

[22] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[23] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management*, 2013. doi:

10.1145/2464157.2464160.

[24] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.

[25] PyCryptodome. Pycryptodome documentation, n.d. URL `https://pycryptodome.readthedocs.io/`. Accessed 2026.

[26] Microsoft. Gpu preemption. Microsoft Learn, n.d.. URL `https://learn.microsoft.com/en-us/windows-hardware/drivers/display/gpu-preemption`. Accessed 2026.

[27] Microsoft. Timeout detection and recovery (tdr). Microsoft Learn, n.d.. URL `https://learn.microsoft.com/en-us/windows-hardware/drivers/display/timeout-detection-and-recovery`. Accessed 2026.